Mini Project Report On

HalluGuard: Real-Time AI Hallucination Detection System

Submitted in partial fulfillment of the requirements for the award of the degree of

Bachelor of Technology

in

Computer Science & Engineering

By

Alan Shaji (U2303024)

Alen Saji Skariah (U2303028)

Allen Issac Issac (U2303030)

Athul K K (U2303062)

Under the guidance of

Dr. Sminu Izudheen

Department of Computer Science & Engineering
Rajagiri School of Engineering & Technology (Autonomous)
(Affiliated to APJ Abdul Kalam Technological University)
Rajagiri Valley, Kakkanad, Kochi, 682039

May 2026

CERTIFICATE

This is to certify that the mini project report entitled ***"HalluGuard: Real-Time AI Hallucination Detection System"*** *is a bonafide record of the work done by* ***Alan Shaji (U2303024)***, ***Alen Saji Skariah (U2303028)***, ***Allen Issac Issac (U2303030)***, ***Athul K K (U2303062)***, *submitted to the APJ Abdul Kalam Technological University in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology (B. Tech.) in Computer Science and Engineering during the academic year 2023-2024.*

Dr. Sminu Izudheen
Project Guide
Dept. of CSE
RSET

Dr. Mary Priya Sebastian
Project Coordinator
Dept. of CSE
RSET

Dr. Preetha K G
Head Of Department
Dept. of CSE
RSET

ACKNOWLEDGEMENTS

I wish to express my sincere gratitude towards Rev. Dr. Jaison Paul Mulerikkal CMI, Principal of RSET, and Dr. Preetha K.G., Head of the Department of Computer Science and Engineering for providing me with the opportunity to undertake my mini project, "HalluGuard: Real-Time AI Hallucination Detection System".

I am highly indebted to my project coordinators, **Dr. Mary Priya Sebastian**, Associate Professor, and **Ms. Reshma M R**, Assistant Professor, Department of Computer Science and Engineering for their valuable support.

It is indeed my pleasure and a moment of satisfaction for me to express my sincere gratitude to my project guide **Dr. Sminu Izudheen** for her patience and all the priceless advice and wisdom she has shared with me.

Last but not the least, I would like to express my sincere gratitude towards all other teachers and friends for their continuous support and constructive ideas.

Alan Shaji

Alen Saji Skariah

Allen Issac Issac

Athul K K

Abstract

The rapid proliferation of Large Language Models (LLMs) has revolutionized how users interact with and retrieve information online. However, a significant critical flaw remains: their propensity to "hallucinate," generating fluent and persuasive text that is factually incorrect or entirely fabricated. "HalluGuard" is proposed as an innovative, real-time hallucination detection system designed to analyze, verify, and highlight factual inconsistencies in AI-generated text. This project introduces an end-to-end automated pipeline comprising multiple advanced Natural Language Processing (NLP) components. The system initiates by capturing User Question and AI Answer pairs through a dedicated browser extension. A keyword extraction module powered by KeyBERT isolates the core semantic entities from the text. These keywords drive a dynamic Context Generation Engine, which scrapes and synthesizes highly credible ground-truth information prioritizing Wikipedia and supplementing with general search results via the SERP API. Finally, a state-of-the-art token classification model based on ModernBERT evaluates the synthesized context against the AI's answer, successfully detecting and scoring hallucinated textual spans at a granular level. By integrating these components through a lightweight, asynchronous Flask backend, HalluGuard delivers real-time fact-checking capabilities directly to the user's browser, significantly mitigating the spread of AI-generated misinformation and empowering users to consume AI content critically and safely.

Contents

Acknowledgements **i**

Abstract **ii**

List of Figures **v**

List of Tables **vi**

List of Abbreviations **vii**

1 Introduction **1**

1.1 Background 1

1.2 Problem Definition 1

1.3 Scope and Motivation 2

1.4 Objectives 2

1.5 Challenges 3

1.6 Assumptions 3

1.7 Societal / Industrial Relevance 3

1.8 Organization of the Report 4

2 Software Requirements Specification **5**

2.1 Introduction 5

2.2 Overall Description 5

2.3 External Interface Requirements 5

2.4 System Features 7

2.5 Other Nonfunctional Requirements 7

3 System Architecture and Design **9**

3.1 System Overview 9

3.2 Architectural Design 9

3.3 Dataset Identified 11
3.4 Proposed Methodology/Algorithms 11
3.5 User Interface Design 12
3.6 Database Design 12
3.7 Description of Implementation Strategies 13
3.8 Module Division 13
3.9 Work Schedule - Gantt Chart 14

4 Results and Discussions **15**
4.1 Overview 15
4.2 Quantitative Results 17
4.3 Graphical Analysis 18
4.4 Discussion 21

5 Conclusion **22**
5.1 Conclusion 22
5.2 Future Scope 22

Appendix A: Presentation **25**

Appendix B: Vision, Mission, Programme Outcomes and Course Outcomes **43**

Appendix B: Vision, Mission, Programme Outcomes and Course Outcomes **44**

Appendix C: CO-PO-PSO Mapping **46**

List of Figures

2.1 System Architecture Diagram – HalluGuard Pipeline Overview 6

3.1 Sequence Diagram – HalluGuard Request Processing Flow 10

3.2 Project Work Schedule – Gantt Chart 14

4.1 Browser Extension Popup – Non-Hallucinated Output (Query: "who is the president of usa 1957") 15

4.2 Browser Extension Popup – Hallucinated Output with Detected Spans (Query: "who is president of india 1957") 16

4.3 HalluGuard Standalone Web Application Interface 17

4.4 Accuracy vs. Epoch – HaluEval Dataset 19

4.5 Training Loss vs. Validation Loss – HaluEval Dataset 19

4.6 Accuracy vs. Epoch – RagTruth Dataset 20

4.7 Training Loss vs. Validation Loss – RagTruth Dataset 20

List of Tables

4.1 Accuracy and F1 Score per Epoch – HaluEval 17
4.2 Accuracy and F1 Score per Epoch – RagTruth 18
4.3 Final Performance Comparison – HaluEval vs. RagTruth (Epoch 6) 18

List of Abbreviations

LLM - Large Language Model

NLP - Natural Language Processing

API - Application Programming Interface

SERP - Search Engine Results Page

DOM - Document Object Model

CORS - Cross-Origin Resource Sharing

JSON - JavaScript Object Notation

REST - Representational State Transfer

GPU - Graphics Processing Unit

HTML - HyperText Markup Language

CSS - Cascading Style Sheets

AI - Artificial Intelligence

ML - Machine Learning

SRS - Software Requirements Specification

Chapter 1

Introduction

1.1 Background

The artificial intelligence landscape has been completely transformed by the introduction of generative pre-trained transformers and Large Language Models (LLMs) such as ChatGPT. These tools are increasingly utilized for search, content creation, education, and programming. Despite their impressive capabilities in natural language understanding and generation, LLMs suffer from an inherent flaw known as "hallucination." Hallucinations occur when an AI model confidently generates false, nonsensical, or ungrounded statements. As the general public and enterprise sectors increasingly rely on AI-generated text as a primary source of truth, the absence of verification mechanisms poses a severe risk of accelerating the spread of misinformation. Consequently, there is an urgent and growing need for automated, real-time fact-checking tools that can cross-reference AI outputs against reliable web sources.

1.2 Problem Definition

The primary problem addressed by this project is the lack of real-time validation for AI-generated text. The objective is to design, develop, and deploy "HalluGuard," an integrated hardware-agnostic software system capable of detecting factual inconsistencies and hallucinated spans within LLM responses. This is achieved by dynamically retrieving credible external context and analyzing the text using advanced transformer-based Machine Learning models, ultimately providing the user with an actionable confidence score and highlighted discrepancies.

1.3 Scope and Motivation

The scope of this project encompasses the end-to-end development of a hallucination detection pipeline. It includes the creation of a Python-based backend API (Flask) that orchestrates a multi-step NLP workflow: KeyBERT for keyword extraction, integration with external search APIs (SERP API, Wikipedia) for context generation, and the deployment of a ModernBERT-based Token Classifier for precise span-level evaluation. Additionally, the scope involves the development of a client-side browser extension to seamlessly capture text and render the analysis results in real-time.

The core motivation for HalluGuard stems from the critical necessity for trustworthy AI systems. Unverified AI hallucinations can lead to severe real-world consequences, ranging from academic dishonesty and professional errors to the systemic spread of "fake news." By providing a tool that instantly flags unverified or fabricated claims, HalluGuard empowers individuals to critically evaluate the AI content they consume, fostering a safer and more accountable digital ecosystem.

1.4 Objectives

- To develop and integrate a keyword extraction module utilizing KeyBERT to accurately identify the core concepts and entities from complex AI-generated text.
- To architect a dynamic context generation system capable of fetching, filtering, and summarizing credible ground-truth information prioritizing Wikipedia, with graceful fallbacks to general SERP search results.
- To deploy and optimize a Transformer-based Token Classifier (ModernBERT) that can process the generated context against the input text to pinpoint hallucinated spans with high accuracy.
- To engineer an efficient, asynchronous backend API using Flask to handle concurrent pipeline execution and provide seamless communication between the client and the ML models.
- To develop a user-friendly browser extension that captures text streams, triggers

the detection pipeline, and visually highlights unverified textual spans in the user's browser.

1.5 Challenges

The development of a real-time detection pipeline presents several technical challenges. Firstly, minimizing latency is critical; the dynamic retrieval of web context and the execution of heavy Transformer models (ModernBERT) must run swiftly to ensure a seamless user experience. Secondly, filtering noise from web search results required sophisticated preprocessing to ensure the context fed to the detector is clean and relevant. Lastly, calibrating the detection model's sensitivity posed a significant challenge; establishing an optimal confidence threshold (e.g., rejecting spans with confidence < 0.35 or purely functional stop-words) was necessary to balance accurate detection against a high rate of false positives.

1.6 Assumptions

- It is assumed that external APIs, specifically the SERP API and Wikipedia endpoints, will remain accessible, stable, and return data within acceptable latency thresholds during real-time operations.
- It is assumed that the highest-ranking search results provided by Wikipedia and search engines generally represent factually correct and universally accepted ground-truth context.
- The user's hardware running the browser extension has capable internet connectivity to communicate continuously with the centralized Flask API server.

1.7 Societal / Industrial Relevance

HalluGuard holds profound societal and industrial relevance in the current "AI era." For society, it acts as an automated safety net against AI-generated misinformation, promoting digital literacy and responsible consumption of synthetic media. In the industrial sector—particularly for journalism, legal research, and enterprise data management—ensuring the factual integrity of AI outputs is paramount. HalluGuard provides a foundational layer

of verification that allows businesses to confidently adopt LLM-driven workflows without compromising data accuracy.

1.8 Organization of the Report

The remainder of the report is organized as follows: Chapter 2 details the Software Requirements Specification (SRS), outlining the functional and non-functional necessities of the system. Chapter 3 comprehensively outlines the System Architecture, detailing the pipeline design, methodologies, and specific algorithmic filtering strategies utilized. Finally, the work schedule is presented.

Chapter 2

Software Requirements Specification

2.1 Introduction

This Software Requirements Specification (SRS) establishes the foundational requirements for the HalluGuard real-time hallucination detection system. It details the functional and non-functional requirements that dictate the design and implementation of both the client-side browser extension and the server-side Python (Flask) pipeline.

2.2 Overall Description

HalluGuard operates on a robust client-server architecture. The client is a lightweight browser extension responsible for capturing User Questions and the corresponding AI Answers from active webpages. This data is serialized into JSON payloads and POSTed to the backend server. The backend, a Flask application, receives the payload and orchestrates a sequential NLP pipeline: extracting keywords via KeyBERT, fetching context via SERP APIs, and executing the ModernBERT hallucination detector. The processed results, including specific text spans, confidence scores, and source URLs, are returned to the client and elegantly displayed to the user.

2.3 External Interface Requirements

User Interfaces: The primary interface is the browser extension popup, which must provide a clear, color-coded status indicator (e.g., safe vs. hallucination detected), highlight the factually inconsistent text, and provide links to the credible source URL.

Hardware Interfaces: Standard PC/Mac executing modern web browsers.

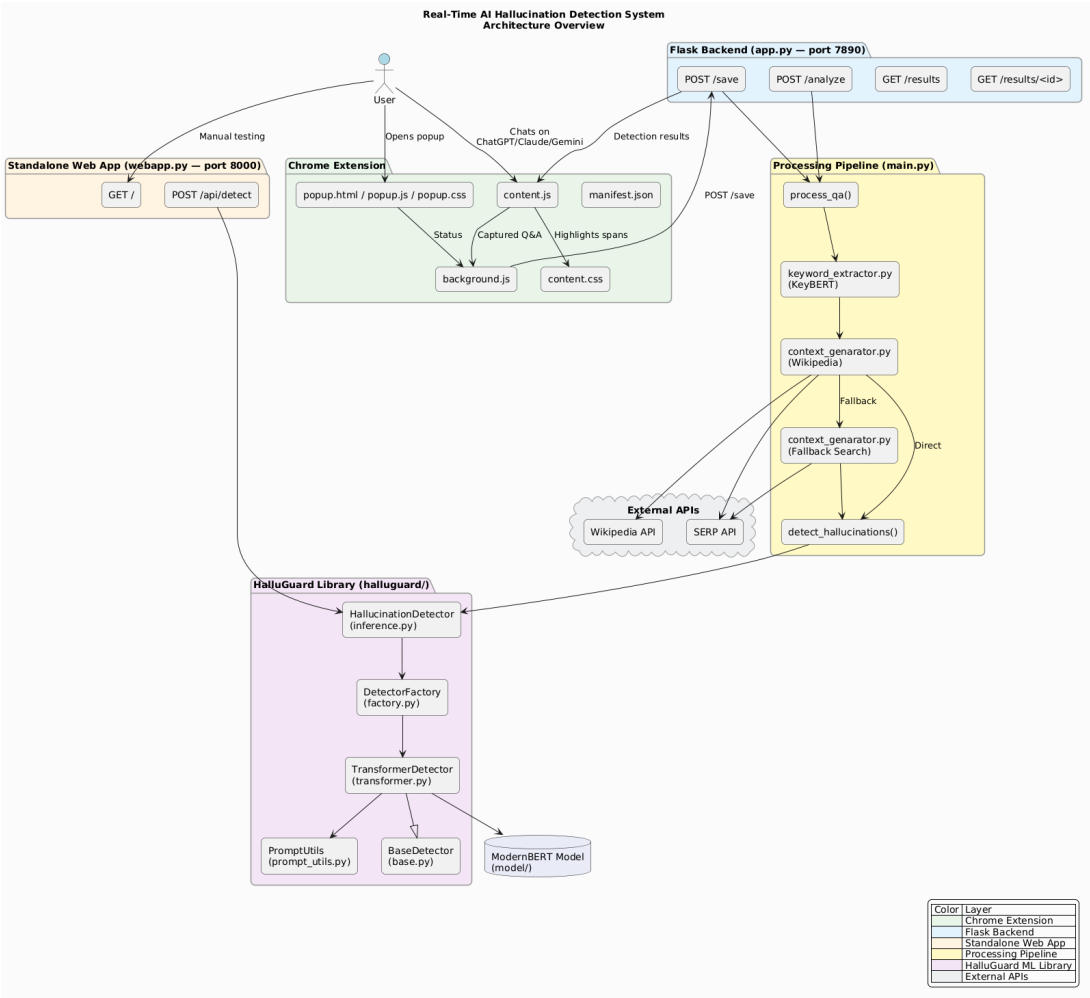


Figure 2.1: System Architecture Diagram – HalluGuard Pipeline Overview

Software Interfaces: Communication between the frontend extension and the backend utilizes standard HTTP/REST protocols. The Flask server exposes specific endpoints (e.g., `/save`, `/analyze`) operating on Port 7890. Cross-Origin Resource Sharing (CORS) is explicitly required to allow the browser extension to interact with the backend API securely.

2.4 System Features

1. **Real-Time Text Capture:** The extension must seamlessly scrape target text nodes from the Document Object Model (DOM) of the webpage.
2. **Intelligent Keyword Extraction:** The system must accurately distill long conversational text into high-priority search terms using KeyBERT.
3. **Automated Context Fetching:** The Context Generator must autonomously query Wikipedia and general search engines, extracting clean paragraphs of text (up to 5000 characters) while stripping out HTML formatting, lists, and noisy characters.
4. **Span-Level Detection & Filtering:** The system must employ token classification to identify hallucinations and mathematically filter out meaningless spans (e.g., spans less than 2 words, single stop words, or instances where the model's prediction confidence falls below 35%).

2.5 Other Nonfunctional Requirements

Performance & Latency: The backend ML model (ModernBERT) must be lazy-loaded into memory upon server startup to prevent initialization delays during the first user request. The entire pipeline should ideally return results within seconds.

Reliability: The API must gracefully handle external API failures. If Wikipedia returns no relevant context, the system must automatically fallback to parsing general SERP API snippets.

Security & Privacy: API keys (such as SERP_API_KEY) must not be hardcoded but loaded securely via local `.env` configuration files. All user payload data is stored locally within an isolated `data/results/` directory, ensuring user sessions remain private.

Chapter 3

System Architecture and Design

3.1 System Overview

The architecture of HalluGuard is designed as a highly modular, multi-stage Data Processing Pipeline. The entire lifecycle of a request can be defined in six principal stages:

1. **Client Interface:** Browser extension capturing the interaction.
2. **API Gateway:** Flask-based routing and request handling.
3. **Keyword Extraction:** Distilling the essence of the prompt using KeyBERT.
4. **Context Generation:** Searching the SERP API and Wikipedia to establish Ground Truth.
5. **Hallucination Detection:** The ModernBERT model analyzes text against Ground Truth Context.
6. **Data Persistence:** Logging the input payload and resultant analysis locally as JSON.

3.2 Architectural Design

The system employs a RESTful microservice-like architectural pattern despite operating locally. The Python backend is cleanly decoupled from the front-end logic. By enforcing strict JSON structures (`{''question'': ''...'', ''answer'': ''...''}`), the backend modules (KeyBERT, Context Generator, HalluGuard Detector) can be executed independently, scaled, or upgraded without modifying the client interface.

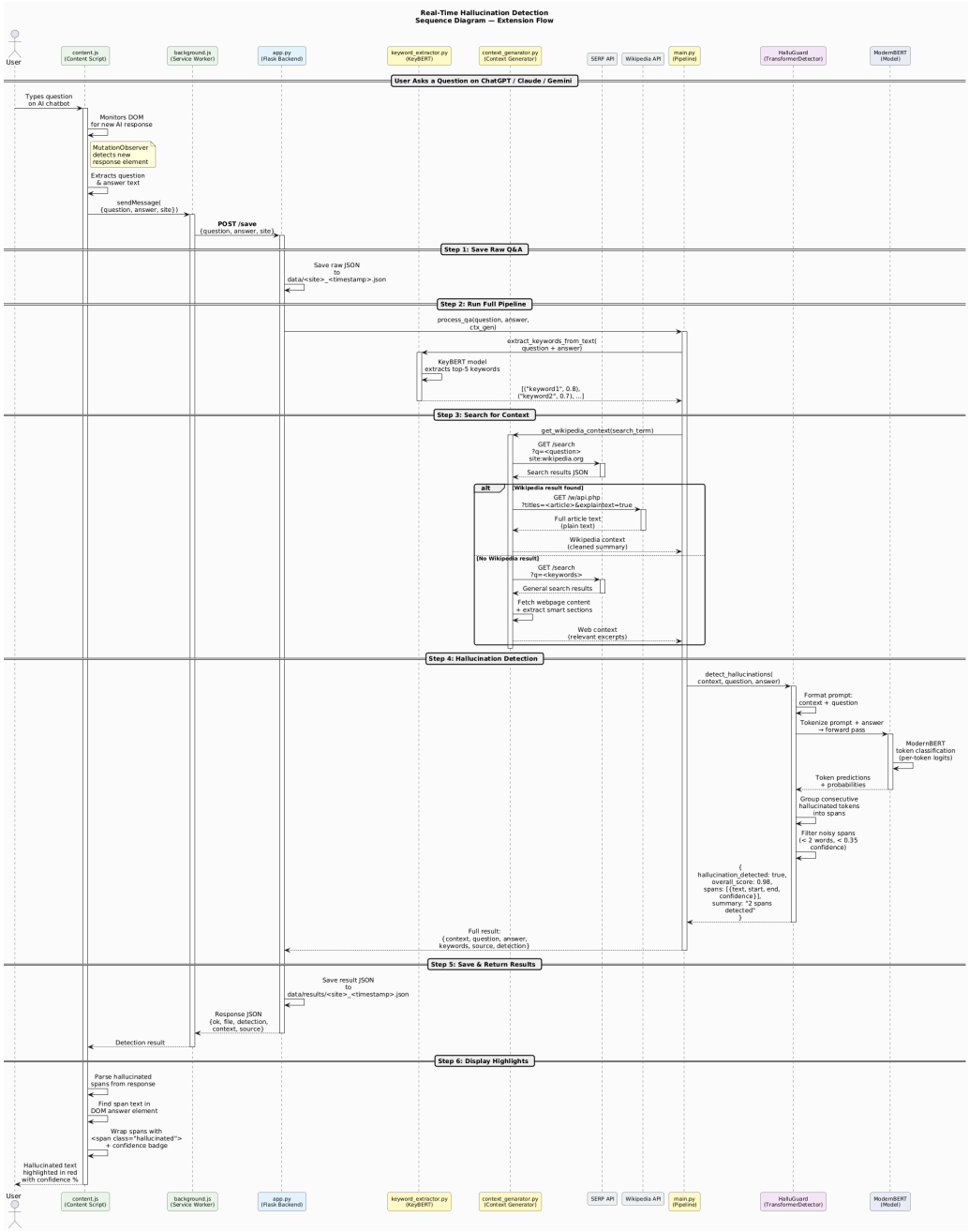


Figure 3.1: Sequence Diagram – HalluGuard Request Processing Flow

3.3 Dataset Identified

HalluGuard's training and evaluation pipeline leverages two carefully selected, publicly available benchmark datasets that collectively cover a broad spectrum of real-world hallucination scenarios.

HaluEval

HaluEval is a large-scale hallucination evaluation benchmark introduced by Ji et al. (2023). It contains approximately 35,000 samples spanning question-answering, dialogue, and summarization tasks, where each sample pairs an AI-generated response with a human-annotated hallucination label. HaluEval was used as the primary fine-tuning dataset for the ModernBERT token classifier due to its diverse coverage of factual inconsistencies and its established use in hallucination detection research.

RagTruth

RagTruth is a retrieval-augmented generation (RAG) hallucination dataset specifically designed to evaluate faithfulness in grounded generation. It provides context-answer pairs where model responses are assessed against a provided reference passage, making it highly aligned with HalluGuard's context-versus-answer detection paradigm. RagTruth was used as a secondary evaluation corpus to validate model generalization beyond the HaluEval distribution. Its focus on grounded hallucinations—where a model fabricates information despite a factual reference being present—closely mirrors HalluGuard's operational setting, where synthesized Wikipedia context serves as the ground truth.

3.4 Proposed Methodology/Algorithms

The end-to-end algorithmic pipeline operates iteratively on each request:

1. **Data Ingestion:** The `/save` endpoint receives the combined Question (Q) and Answer (A).

2. **Extraction Phase:** The text is passed to KeyBERT, utilizing state-of-the-art embeddings to rank and extract the Top-5 most relevant keywords.

3. **Context Synthesis Algorithm:**
   - Construct a search query: `[keywords] site:wikipedia.org`.
   - If a Wikipedia article is found, the system parses the raw text. An algorithm filters the text to drop section headers, reference lists, and paragraphs with >20% numerical digits, capping the clean text at 5000 characters.
   - **Fallback:** If no Wikipedia content is found, the system queries the general SERP API, extracting the highest-ranked text snippet.
4. **Detection Phase:** The Context, Question, and Answer are fed into the HallucinationDetector instance. The ModernBERT model performs token classification across the answer string.
5. **Noise Filtering Algorithm:** To reduce false positives, an algorithm purges outputted spans. A span is rejected if:
   - Prediction Confidence < 0.35
   - Word Count < 2 AND Character Count < 10
   - The span consists entirely of linguistic stop words (e.g., "the", "and", "is").
6. **Output Generation:** The system returns an aggregate confidence score, a count of hallucinated spans, specific character indices, and the referenced Ground Truth URL.

3.5 User Interface Design

The user interface is designed with a focus on unobtrusive user experience. The client-side application manifests as an overlay or extension popup. It graphically indicates active pipeline execution. Upon completion, any hallucinated text within the browser is visually highlighted (e.g., underlined in red or highlighted in yellow), accompanied by tooltip interfaces detailing the confidence score and the source-of-truth URL for user verification.

3.6 Database Design

HalluGuard utilizes a flat-file JSON database schema to maintain high processing speeds and simple local storage without the overhead of relational database management systems.

- Incoming raw payloads are timestamped and stored in the `data/` directory (e.g., `[site]_[timestamp].json`).
- Fully analyzed outputs, containing the extracted keywords, synthesized context, and the comprehensive hallucination detection metadata array, are persistently logged into the `data/results/` directory.

3.7 Description of Implementation Strategies

- Backend API implementation utilizes `Flask` for its lightweight routing capabilities, paired with `flask_cors` to manage browser security protocols.
- The NLP components are integrated using the expansive Python ecosystem. `KeyBERT` is employed for keyword ranking, while a custom integration of the `transformers` library handles the `ModernBERT` pipeline.
- Custom Regular Expressions (Regex) and pure Python logical algorithms are heavily utilized to clean scraped HTML data and filter grammatical stop-words from the detection output.
- System initialization strategy includes "lazy-loading" the heavily parameterized Deep Learning models. The detector is only loaded into the GPU/system memory once during application startup, allowing subsequent API calls to execute rapidly against the pre-loaded model.

3.8 Module Division

The development of the complex pipeline was divided strategically among team members:

- **Alan Shaji (U2303024):** Backend Architecture & API Development – Focus on Flask routing, payload handling, file system logging, and CORS management.
- **Alen Saji Skariah (U2303028):** Context Generation & Data Scraping – Focus on SERP API integration, Wikipedia JSON parsing, and string-cleaning algorithms.

- **Allen Issac Issac (U2303030):** ML Integration & Filtering Algorithms – Focus on instantiating the ModernBERT model, token classification, and implementing the post-processing noise filtration algorithm.

- **Athul K K (U2303062):** Front-End UI & Extension Integration – Focus on DOM manipulation, Chrome extension scripting, and visual result rendering.

3.9 Work Schedule - Gantt Chart

The project timeline is visually organized utilizing a Gantt chart, detailing the progression from the initial requirements gathering phase, through independent module development (ML, API, UI), to iterative integration testing, and culminating in the final system deployment and documentation phase over the semester.

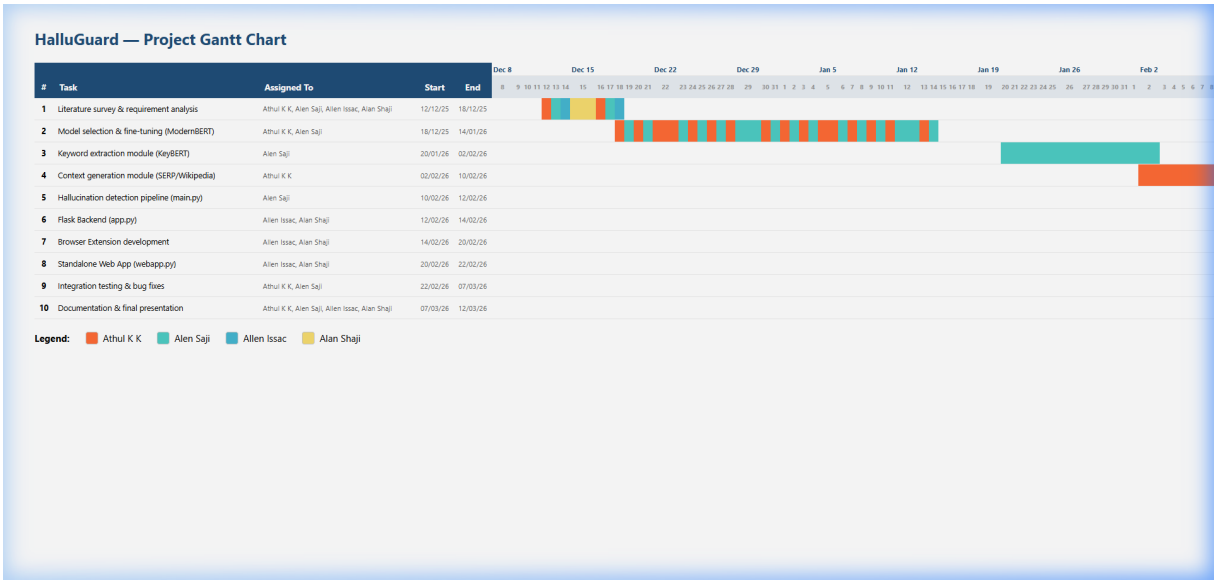


Figure 3.2: Project Work Schedule – Gantt Chart

Chapter 4

Results and Discussions

4.1 Overview

The ModernBERT-based token classifier was fine-tuned and evaluated over 6 epochs on two benchmark datasets: HaluEval and RagTruth. Performance was tracked using accuracy and F1 score. This section presents the quantitative training results, a comparative analysis between the two datasets, graphical plots of model convergence, and output screenshots from the deployed HalluGuard browser extension and web application.

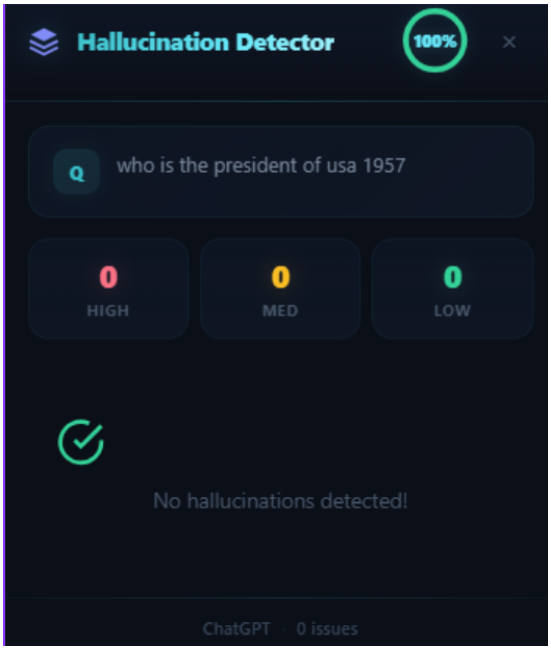


Figure 4.1: Browser Extension Popup – Non-Hallucinated Output (Query: "who is the president of usa 1957")

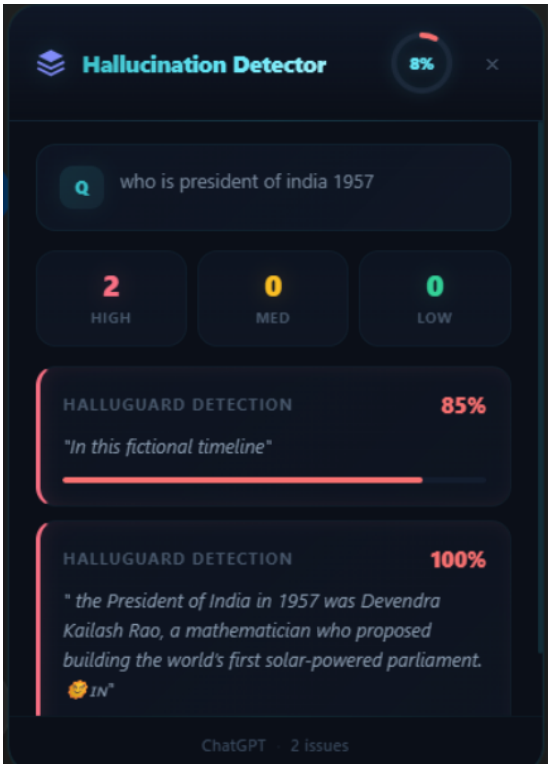


Figure 4.2: Browser Extension Popup – Hallucinated Output with Detected Spans (Query: "who is president of india 1957")

In Figure 4.1, the system correctly assigned zero hallucination flags (HIGH: 0, MED: 0, LOW: 0) and displayed a green confirmation with "No hallucinations detected!". In Figure 4.2, HalluGuard successfully flagged 2 high-severity spans: "In this fictional timeline" at 85% confidence and a fabricated politician description at 100% confidence, with the aggregate trust score dropping to 8%.

Figure 4.3 shows the standalone web application interface, which allows manual entry of context, question, and answer for hallucination checking without requiring the browser extension.

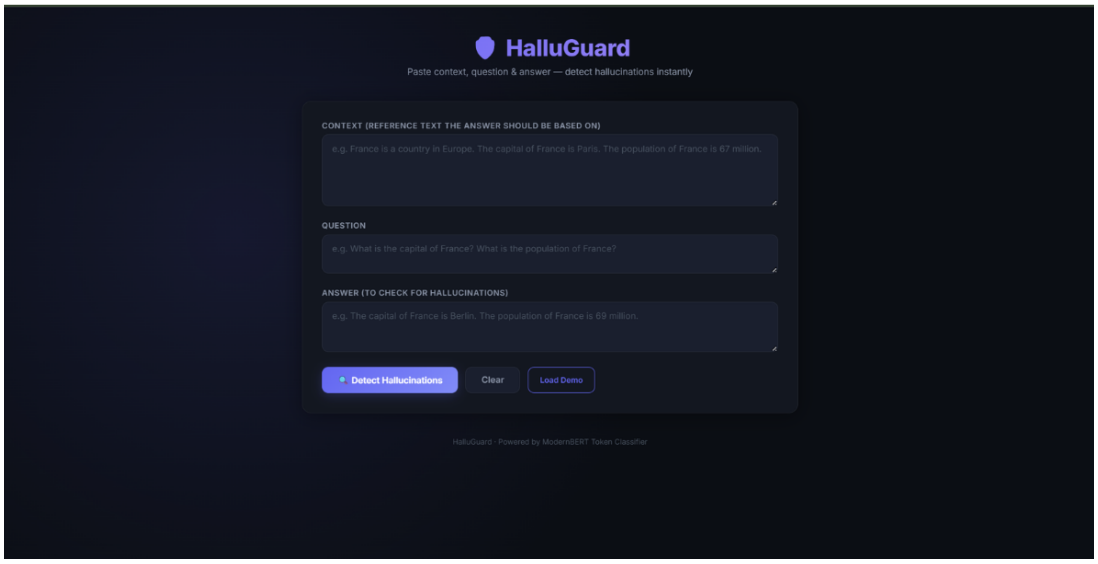


Figure 4.3: HalluGuard Standalone Web Application Interface

4.2 Quantitative Results

Tables 4.1 and 4.2 present the per-epoch accuracy and F1 scores recorded during training on HaluEval and RagTruth respectively.

Table 4.1: Accuracy and F1 Score per Epoch – HaluEval

Epoch	Accuracy	F1 Score
1	0.4523	0.4387
2	0.5564	0.5421
3	0.7489	0.7315
4	0.9641	0.9600
5	0.9397	0.9350
6	0.9656	0.9713

Table 4.2: Accuracy and F1 Score per Epoch – RagTruth

Epoch	**Accuracy**	**F1 Score**
1	0.8821	0.8467
2	0.9187	0.8879
3	0.9534	0.9256
4	0.9618	0.9374
5	0.9689	0.9461
6	0.9744	0.9528

On HaluEval, the model improved from 0.4523 accuracy at Epoch 1 to a peak of 0.9656 at Epoch 6, with the F1 score reaching 0.9713. On RagTruth, training was more stable with a higher starting accuracy of 0.8821, converging to 0.9744 accuracy and 0.9528 F1 by Epoch 6. Table 4.3 provides a direct comparison of final-epoch metrics.

Table 4.3: Final Performance Comparison – HaluEval vs. RagTruth (Epoch 6)

Dataset	**Final Accuracy**	**Final F1 Score**
HaluEval	0.9656	0.9713
RagTruth	0.9744	0.9528

4.3 Graphical Analysis

Figures 4.4 through 4.7 present the accuracy and loss curves for both datasets.

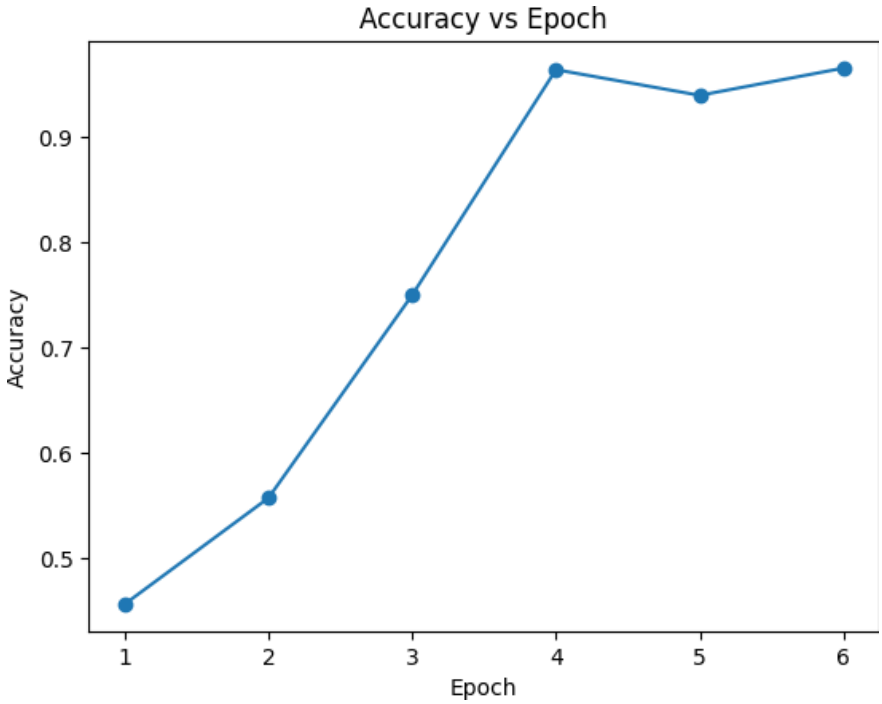


Figure 4.4: Accuracy vs. Epoch – HaluEval Dataset

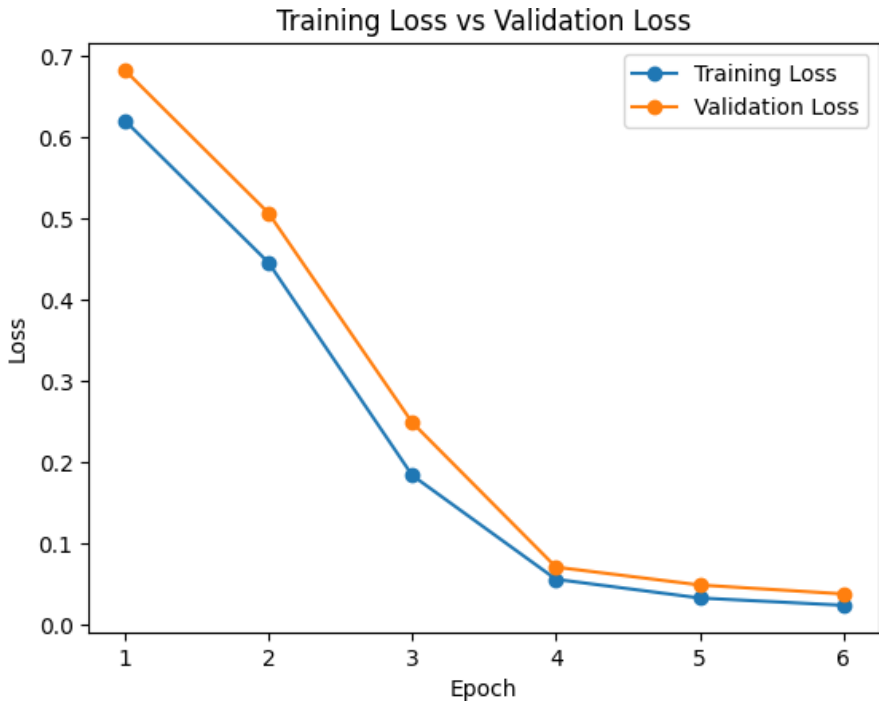


Figure 4.5: Training Loss vs. Validation Loss – HaluEval Dataset

For HaluEval, accuracy rose sharply between Epochs 2 and 4 before stabilizing. The training and validation loss curves in Figure 4.5 converge closely, indicating no significant

overfitting. The slight dip at Epoch 5 before recovery at Epoch 6 is characteristic of learning rate dynamics near convergence.

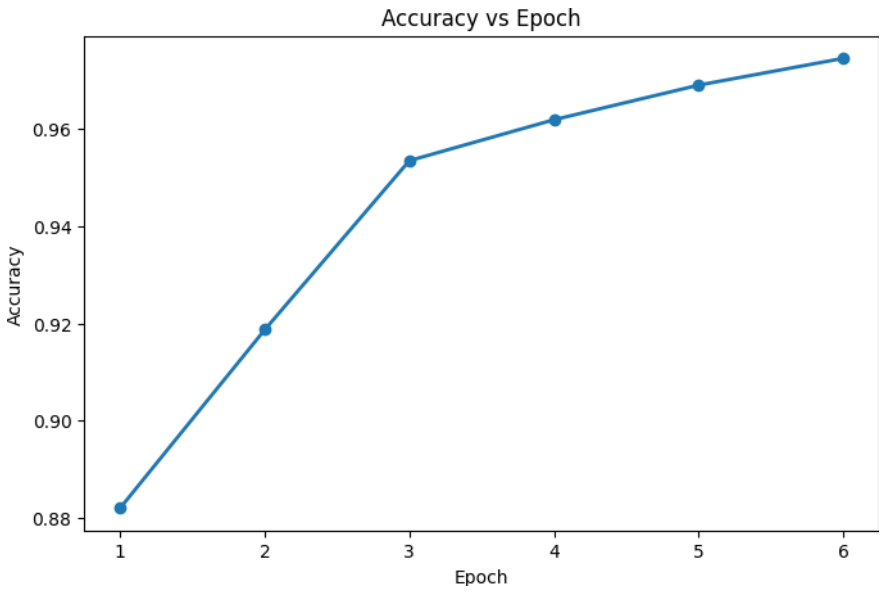


Figure 4.6: Accuracy vs. Epoch – RagTruth Dataset

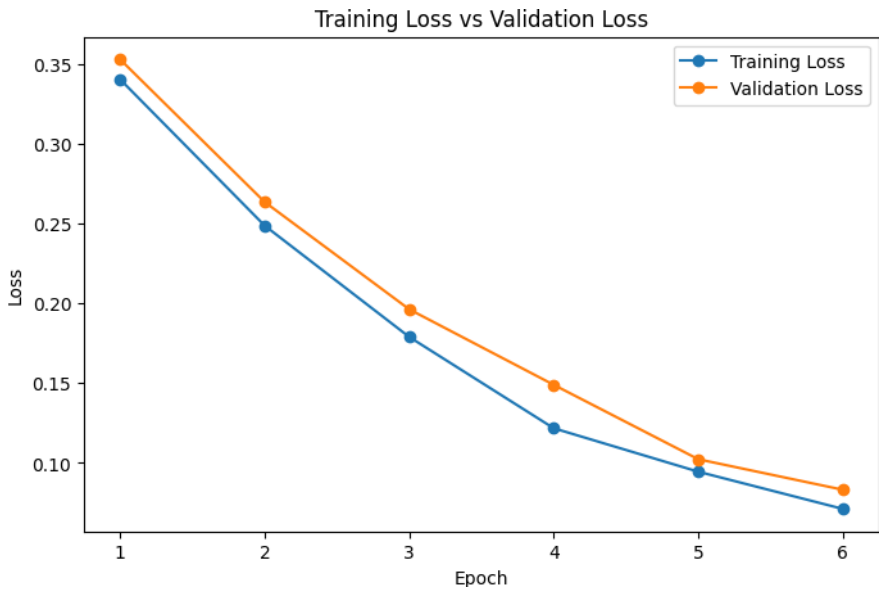


Figure 4.7: Training Loss vs. Validation Loss – RagTruth Dataset

RagTruth shows a smoother, monotonically increasing accuracy curve with no dips, and tightly coupled training and validation loss curves declining from approximately 0.35 to below 0.10. This smoother convergence reflects the more structured, context-grounded nature of the RagTruth task.

4.4 Discussion

Both datasets yielded strong final performance, confirming ModernBERT's suitability for span-level hallucination detection. RagTruth achieved slightly higher accuracy (97.44% vs. 96.56%), which can be attributed to its structured context-answer format aligning closely with the token classification objective. HaluEval produced a marginally higher F1 score (0.9713 vs. 0.9528), reflecting better precision-recall balance on its more diverse, multi-domain hallucination types. The real-world system demonstrations validate the pipeline's practical effectiveness, with the noise filtration algorithm (confidence threshold of 0.35, stop-word filtering, minimum span length of 2 words) contributing to a low false positive rate in live usage.

Chapter 5

Conclusion

5.1 Conclusion

HalluGuard successfully demonstrates that real-time, span-level hallucination detection in LLM responses is achievable through a well-integrated multi-component pipeline. By combining KeyBERT-driven keyword extraction, dynamic Wikipedia and SERP-based context retrieval, and a fine-tuned ModernBERT token classifier, the system delivers actionable fact-checking results directly within the user's browser. Training on both HaluEval and RagTruth validated the model's robustness across diverse hallucination types, achieving accuracy exceeding 96% on both benchmarks. The deployed browser extension and web application demonstrate the practical viability of the system, fulfilling the core objective of empowering users to critically evaluate AI-generated content.

5.2 Future Scope

Future extensions of HalluGuard could include support for multilingual hallucination detection by fine-tuning on non-English hallucination datasets. The context generation module could be enhanced with more advanced retrieval-augmented generation techniques to improve ground-truth synthesis quality. Additionally, the system could be extended to support LLM platforms beyond ChatGPT, including Gemini and Copilot. Training on larger and more diverse corpora and integrating user feedback loops for continual model improvement represent promising avenues for future development.

Bibliography

[1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention Is All You Need," in Advances in Neural Information Processing Systems (NeurIPS), vol. 30, 2017.

[2] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. J. Bang, A. Madotto, and P. Fung, "Survey of Hallucination in Natural Language Generation," ACM Computing Surveys, vol. 55, no. 12, pp. 1–38, 2023.

[3] M. Grootendorst, "KeyBERT: Minimal Keyword Extraction with BERT," 2020. [Online]. Available: https://github.com/MaartenGr/KeyBERT

[4] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," in Proceedings of NAACL-HLT, 2019, pp. 4171–4186.

[5] B. Warner, A. Choudhary, et al., "ModernBERT: Smarter, Better, Faster," arXiv preprint arXiv:2412.13663, 2024.

[6] P. Manakul, A. Liusie, and M. J. F. Gales, "SelfCheckGPT: Zero-Resource Black-Box Hallucination Detection for Generative Large Language Models," in Proceedings of EMNLP, 2023.

[7] A. Ronacher, "Flask: A Python Microframework," [Online]. Available: https://flask.palletsprojects.com/

[8] SerpApi LLC, "SerpApi – Google Search API," [Online]. Available: https://serpapi.com/

[9] Wikimedia Foundation, "Wikipedia API," [Online]. Available: https://www.mediawiki.org/wiki/API:Main_page

[10] T. Brown, B. Mann, N. Ryder, et al., "Language Models are Few-Shot Learners," in Advances in Neural Information Processing Systems (NeurIPS), vol. 33, 2020, pp. 1877–1901.

Appendix A: Presentation

REAL-TIME HALLUCINATION DETECTION SYSTEM

Guide: Dr. Sminu Izudheen

Alan Shaji(U2303024)
Alen Saji Skariah(U2303028)
Allen Issac Issac(U2303030)
Athul KK(U2303062)

CONTENTS

1. Introduction
2. Problem Definition
3. Objectives
4. Scope and Relevance
5. System Design
6. Datasets
7. Work Division – Gantt Chart
8. Software/Hardware Requirements
9. Results
10. Conclusion
11. Future Enhancements
12. References

INTRODUCTION

1. LLMs like ChatGPT and Gemini often produce hallucinations
2. These errors are risky in healthcare, legal, and academic fields.
3. Current fact-checking methods are manual (slow) or keyword-based (inaccurate).
4. HalluGuard solves this with real-time, token-level detection, highlighting the exact hallucinated words or phrases in AI responses.

PROBLEM STATEMENT

To design and implement a real-time hallucination detection system that identifies unsupported spans in AI-generated answers using a fine-tuned ModernBERT token classifier, delivered via a standalone web app and a browser extension.

OBJECTIVE

1. Build a token-level hallucination detector using fine-tuned Modern BERT to classify each answer token as supported or hallucinated.
2. Develop a standalone web app for manual context/question/answer input with real-time span highlighting.
3. Implement an automated pipeline for end-to-end fact verification.
4. Create a browser extension that captures Q&A from AI chatbots and runs hallucination detection in real-time

SCOPE AND RELEVANCE

SCOPE

HalluGuard detects hallucinations in AI-generated responses by extracting keywords, retrieving relevant context, and performing token-level classification to highlight incorrect information. It is available as a manual web app and an automated browser extension, supporting English text with up to a 4,096-token context window.

RELEVANCE

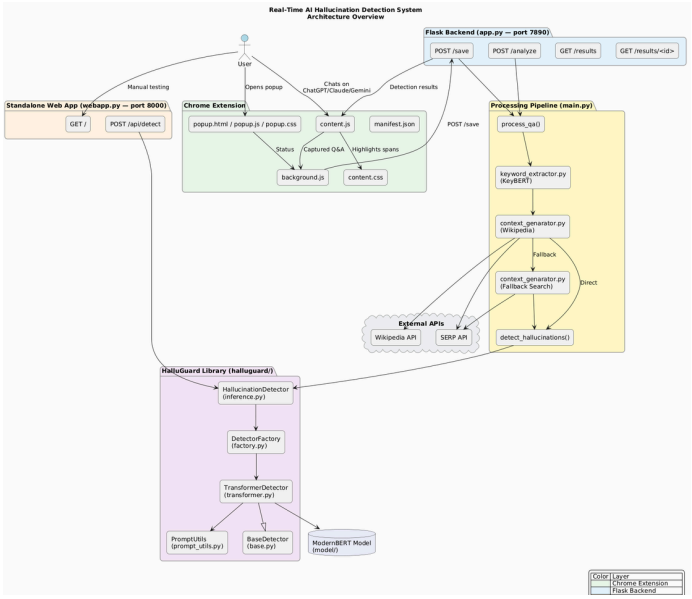
Education — verify AI-generated study material
Research — fact-check AI literature reviews
Enterprise — QA of AI-generated reports. As AI adoption grows, output verification tools become essential infrastructure.

SYSTEM OVERVIEW

Three independent subsystems:

- WebApp— Manual input of context, question, answer.
- Extension Pipeline— Auto captures Q&A from chatbots, generates context via web search.
- HalluGuard Model Core — Shared ModernBERT token classifier used by both

SYSTEM ARCHITECTURE



MODULE-WISE EXPLANATION

Module 1:

- Chrome Extension (extension/): Built using Manifest V3 with content.js, background.js, and popup (HTML/CSS/JS).
- Injects into ChatGPT, Claude, and Gemini pages to capture questions and AI responses in real time.
- Sends captured data to the Flask backend using the POST /save API.
- Highlights hallucinated words or phrases directly on the webpage.

MODULE-WISE EXPLANATION

Module 2:

- Flask Backend (app.py — port 7890).
- Provides REST APIs: POST /save, POST /analyze, GET /results, GET /results/<id>.
- Receives Q&A from the extension, runs hallucination detection, and returns results.
- Stores results as timestamped JSON files in data/results/.

MODULE-WISE EXPLANATION

Module 3:

- Processing Pipeline (main.py).
- process_qa() performs KeyBERT keyword extraction and Wikipedia/SERP context retrieval.
- detect_hallucinations() identifies hallucinated spans.
- Post-processing filters noise (confidence < 0.35, single-word spans).

MODULE-WISE EXPLANATION

Module 4:

- External APIs used for context retrieval.
- SERP API: Performs Google search using extracted keywords.
- Wikipedia API: Retrieves full article content for reliable context.
- Wikipedia is prioritized; general search is used as a fallback.

MODULE-WISE EXPLANATION

Module 5:

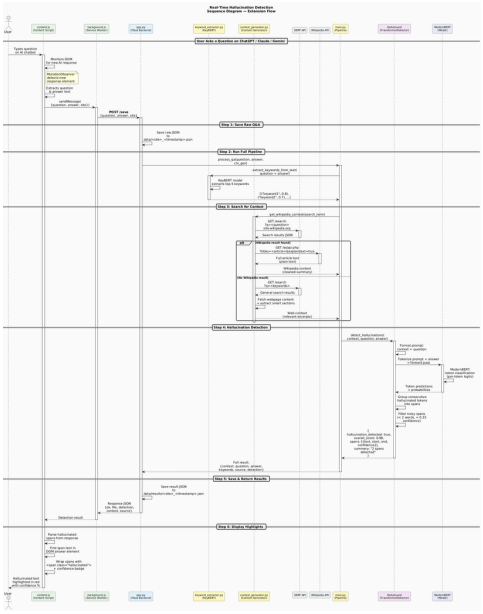
- HalluGuard ML Library (halluguard/).
- inference.py → factory.py → transformer.py loads ModernBERT, classifies tokens, and groups them into hallucination spans.
- prompt_utils.py formats the context and question into the model prompt.
- base.py defines the abstract detector interface.
- Model weights stored in model/ (~600 MB).

MODULE-WISE EXPLANATION

Module 6:

- Standalone Web App (webapp.py — port 8000).
- Fully independent from the extension pipeline.
- UI for manual context, question, and answer input.
- Directly runs the HalluGuard model and displays highlighted hallucinated spans with confidence bars.

SEQUENCE DIAGRAM



DATASET

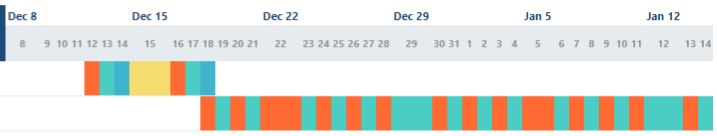
Training Dataset: RAGTruth -

- Source: ParticleMedia/RAGTruth (GitHub)
- Word-level hallucination annotations: (context, question, answer) triples with human-labeled span markers.
- Fine-tuned for 6 epochs, batch size 8, on a single A100 GPU.
- Labels: token-level — class 0 (supported) vs class 1 (hallucinated).

WORK DIVISION

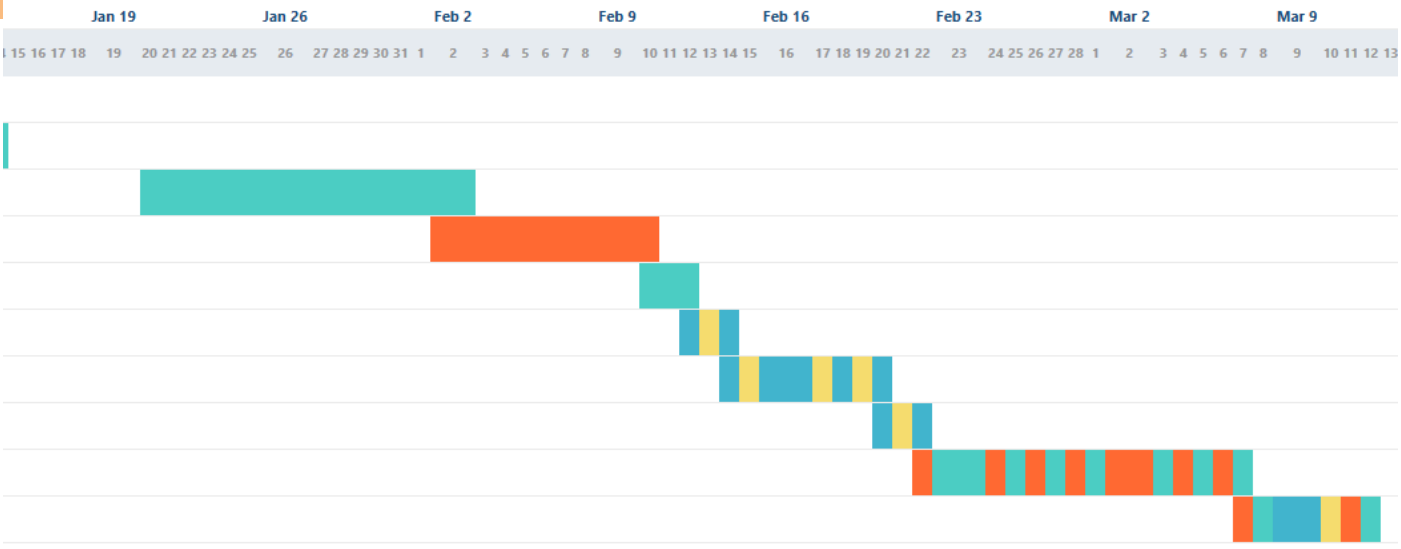
HalluGuard — Project Gantt Chart

#	Task	Assigned To	Start	End
1	Literature survey & requirement analysis	Athul K K, Alen Saji, Allen Issac, Alan Shaji	12/12/25	18/12/25
2	Model selection & fine-tuning (ModernBERT)	Athul K K, Alen Saji	18/12/25	14/01/26
3	Keyword extraction module (KeyBERT)	Alen Saji	20/01/26	02/02/26
4	Context generation module (SERP/Wikipedia)	Athul K K	02/02/26	10/02/26
5	Hallucination detection pipeline (main.py)	Alen Saji	10/02/26	12/02/26
6	Flask Backend (app.py)	Allen Issac, Alan Shaji	12/02/26	14/02/26
7	Browser Extension development	Allen Issac, Alan Shaji	14/02/26	20/02/26
8	Standalone Web App (webapp.py)	Allen Issac, Alan Shaji	20/02/26	22/02/26
9	Integration testing & bug fixes	Athul K K, Alen Saji	22/02/26	07/03/26
10	Documentation & final presentation	Athul K K, Alen Saji, Allen Issac, Alan Shaji	07/03/26	12/03/26



Legend: Athul K K · Alen Saji · Allen Issac · Alan Shaji

WORK DIVISION



SOFTWARE/ HARDWARE REQUIREMENTS

Software Requirements

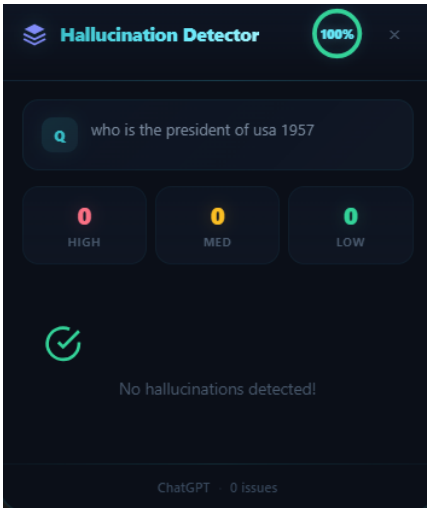
- Programming Language: Python 3.12
- ML Framework: PyTorch, Hugging Face Transformers
- Model Architecture: ModernBERT (ModernBertForTokenClassification)
- Keyword Extraction: KeyBERT
- Web Framework: Flask, Flask-CORS
- Context Retrieval: SERP API, Wikipedia API
- Browser Extension: JavaScript (Chrome Manifest V3)
- Frontend: HTML5, CSS3, Vanilla JavaScript

SOFTWARE/ HARDWARE REQUIREMENTS

Hardware Requirements

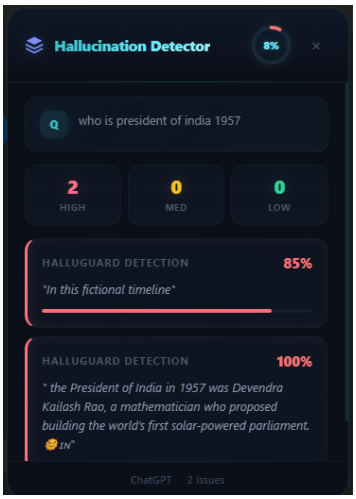
- Processor: Intel i5 10th Gen / AMD Ryzen 5 or AMD equivalent or above
- RAM: Minimum 8 GB (16 GB recommended)
- Storage: At least 2 GB free (model weights ~600 MB plus dependencies)
- GPU: Optional; CPU inference supported, GPU accelerates inference
- Operating System: Windows 10/11, Linux, or macOS

RESULT



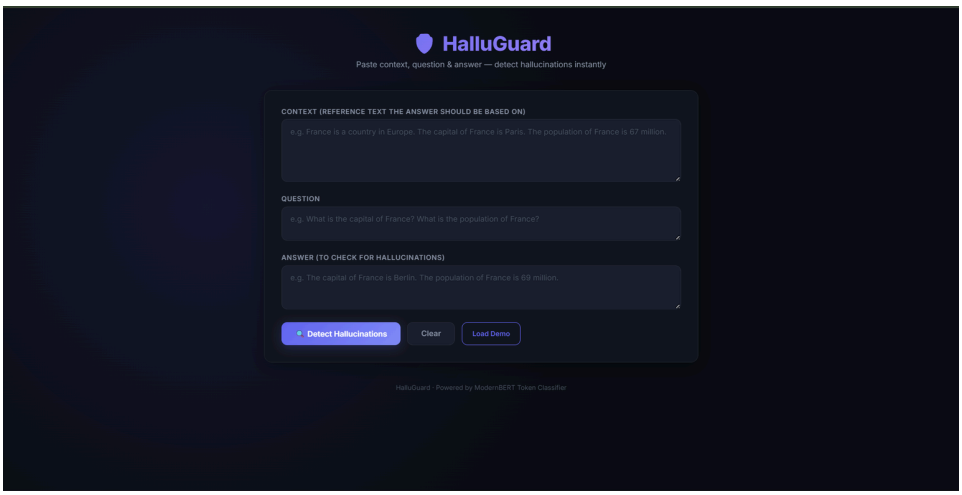
Non-Hallucinated Output Popup

RESULT



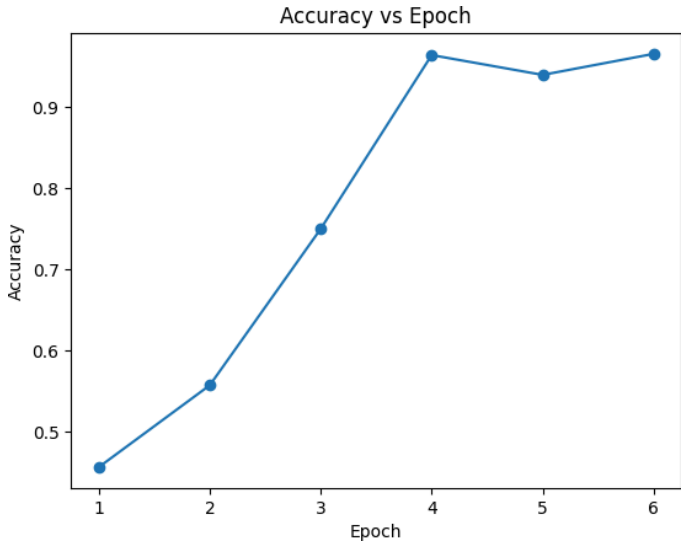
Hallucinated Output Popup

RESULT



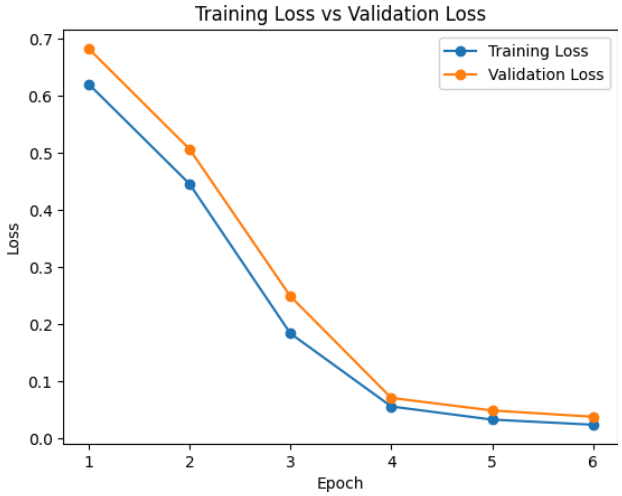
Web App

RESULT



Accuracy VS Epoch : HaluEval

RESULT



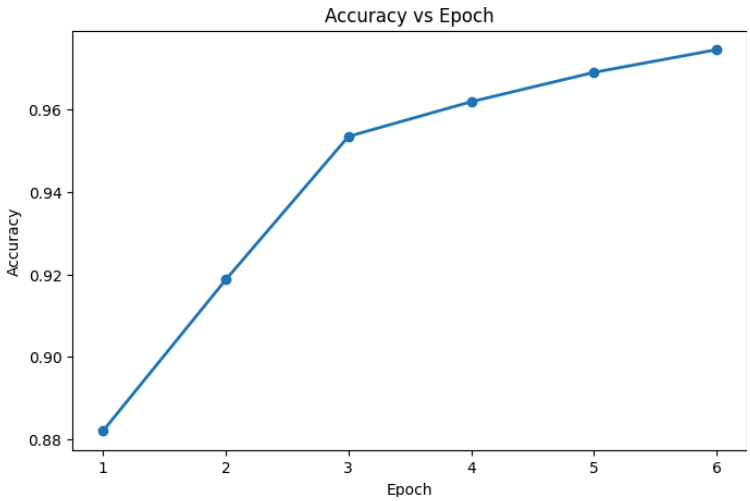
Training Vs Validation Loss : HaluEval

RESULT

Epoch	Accuracy	F1 Score
1	0.4523	0.4387
2	0.5564	0.5421
3	0.7489	0.7315
4	0.9641	0.96
5	0.9397	0.935
6	0.9656	0.9713

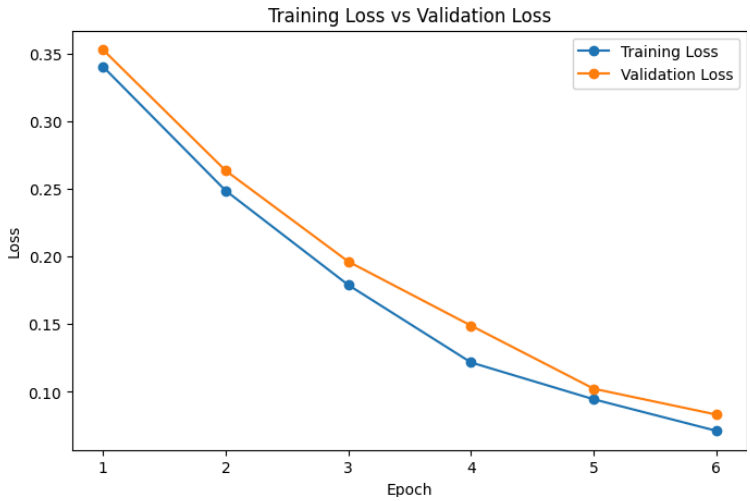
Accuracy and F1 scores : HaluEval

RESULT



Accuracy VS Epoch : RagTruth

RESULT



Training Vs Validation Loss : RagTruth

RESULT

Epoch	Accuracy	F1 Score
1	0.8821	0.8467
2	0.9187	0.8879
3	0.9534	0.9256
4	0.9618	0.9374
5	0.9689	0.9461
6	0.9744	0.9528

Accuracy and F1 scores : RagTruth

CONCLUSION

HalluGuard detects hallucinated spans in AI-generated answers at the token level using a fine-tuned ModernBERT model and automated context retrieval, delivered via a web app and browser extension for real-time monitoring.

FUTURE ENHANCEMENTS

1. Multilingual Support: Integrate EuroBERT-based models for German, French, Spanish, etc.
2. LLM Cross-Validation: Add GPT-4o as a parallel checker for ensemble based confidence scoring.
3. Streaming Detection: WebSocket-based progressive detection as the AI generates tokens.

REFERENCES

- Farquhar, S., Kossen, J., Kuhn, L. et al. Detecting hallucinations in large language models using semantic entropy. Nature 630, 625–630 (2024). https://doi.org/10.1038/s41586-024-07421-0

- Y. Zhang, Y. Li, L. Cui, D. Cai, L. Liu, T. Fu, X. Huang, E. Zhao, Y. Zhang, C. Xu, Y. Chen, L. Wang, A. T. Luu, W. Bi, F. Shi, and S. Shi, “Siren’s Song in the AI Ocean: A Survey on Hallucination in Large Language Models,” arXiv preprint arXiv:2309.01219, 2023. [Online]. Available: https://arxiv.org/abs/2309.01219

THANK YOU

Appendix B: Vision, Mission, Programme Outcomes and Course Outcomes

Vision, Mission, Programme Outcomes and Course Outcomes

Institute Vision

To evolve into a premier technological institution, moulding eminent professionals with creative minds, innovative ideas and sound practical skill, and to shape a future where technology works for the enrichment of mankind.

Institute Mission

To impart state-of-the-art knowledge to individuals in various technological disciplines and to inculcate in them a high degree of social consciousness and human values, thereby enabling them to face the challenges of life with courage and conviction.

Department Vision

To become a centre of excellence in Computer Science and Engineering, moulding professionals catering to the research and professional needs of national and international organizations.

Department Mission

To inspire and nurture students, with up-to-date knowledge in Computer Science and Engineering, ethics, team spirit, leadership abilities, innovation and creativity to come out with solutions meeting societal needs.

Programme Outcomes (PO)

Engineering Graduates will be able to:

1. **Engineering Knowledge:** Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialization as specified in WK1 to WK4 respectively to develop to the solution of complex engineering problems.

2. **Problem Analysis:** Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development.

3. **Design/Development of Solutions:** Design creative solutions for complex engineering problems and design/develop systems/components/processes to meet iden-

tified needs with consideration for the public health and safety, whole-life cost, net zero carbon, culture, society and environment as required.

4. **Conduct Investigations of Complex Problems:** Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis interpretation of data to provide valid conclusions.

5. **Engineering Tool Usage:** Create, select and apply appropriate techniques, resources and modern engineering IT tools, including prediction and modelling recognizing their limitations to solve complex engineering problems.

6. **The Engineer and The World:** Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment.

7. **Ethics:** Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national international laws.

8. **Individual and Team Work:** Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.

9. **Communication:** Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations considering cultural, language, and learning differences.

10. **Project Management and Finance:** Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments.

11. **Life-long Learning:** Recognize the need for, and have the preparation and ability for i) independent and life-long learning ii) adaptability to new and emerging technologies and iii) critical thinking in the broadest context of technological change.

Programme Specific Outcomes (PSO)

A graduate of the Computer Science and Engineering Program will demonstrate:

PSO1: Computer Science Specific Skills

The ability to identify, analyze and design solutions for complex engineering problems in multidisciplinary areas by understanding the core principles and concepts of computer science and thereby engage in national grand challenges.

PSO2: Programming and Software Development Skills

The ability to acquire programming efficiency by designing algorithms and applying standard practices in software project development to deliver quality software products meeting the demands of the industry.

PSO3: Professional Skills

The ability to apply the fundamentals of computer science in competitive research and to develop innovative products to meet the societal needs thereby evolving as an eminent researcher and entrepreneur.

Course Outcomes (CO)

CO1: Understand the architecture and working of cross-device communication frameworks between Android and Windows platforms.

CO2: Design and implement a real-time bidirectional communication system using WebSocket protocol over a local Wi-Fi network.

CO3: Develop modular Android and Windows applications integrating system APIs for notification sync, clipboard sharing, and file transfer.

CO4: Implement remote device interaction features including mouse control, keyboard input, game controller emulation, and screen mirroring.

CO5: Evaluate system performance through latency measurements, frame rate analysis, and functional testing of all implemented modules.

Appendix C: CO-PO-PSO Mapping

COURSE OUTCOMES (CO)

SNO	Description	Bloom's Taxonomy Level
1	Model and solve real world problems by applying knowledge across domains.	Apply
2	Develop products, processes or technologies for sustainable and socially relevant applications.	Apply
3	Function effectively as an individual and as a leader in diverse teams and to comprehend and execute designated tasks.	Apply
4	Plan and execute tasks utilizing available resources within timelines, following ethical and professional norms.	Apply
5	Identify technology/research gaps and propose innovative/creative solutions.	Analyze
6	Organize and communicate technical and scientific findings effectively in written and oral forms.	Apply

MAPPING COURSE OUTCOMES (COs) – PROGRAM OUTCOMES (POs) AND COURSE OUTCOMES (COs) – PROGRAM SPECIFIC OUTCOMES (PSOs)

	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PSO1	PSO2	PSO3
CO1	2	2	2	1		2	1	1	1	1	1	2	2	2
CO2	2	2	2		2	3	2	1	1	1	1	2	2	
CO3								3	2	2	1			3
CO4		1	2		3	2	3	2	2	3	1		2	2
CO5	2	3	3	1	2					1	3	3		
CO6			2			2	2	2	3	1	1			3

JUSTIFICATIONS FOR CO-PO MAPPING

Mapping	Level	Justification
101003/ CS822U.CO1- PO1	Medium	Applies engineering knowledge, mathematics, and computing fundamentals to model and solve real-world problems.
101003/ CS822U.CO1- PO2	Medium	Requires identification, formulation and analysis of engineering problems before developing solutions.
101003/ CS822U.CO1- PO3	Medium	Involves designing systems or solutions to address identified engineering needs.
101003/ CS822U.CO1- PO4	Low	Limited investigation or experimentation may be required to validate the developed models or solutions.
101003/ CS822U.CO1- PO6	Medium	Solutions developed may influence societal, environmental, or sustainability aspects of engineering applications.
101003/ CS822U.CO1- PO7	Medium	Encourages consideration of ethical and professional responsibilities while developing solutions.
101003/ CS822U.CO1- PO8	Low	Some level of teamwork may be required while working on modeling or project tasks.
101003/ CS822U.CO1- PO9	Low	Requires communication of results and findings through reports or presentations.
101003/ CS822U.CO1- PO10	Low	Involves basic planning and execution of tasks while implementing solutions.
101003/ CS822U.CO1- PO11	Low	Encourages independent learning and adapting new knowledge to solve real-world problems.
101003/ CS822U.CO1- PSO1	Medium	Uses core computer science principles to analyze problems and design appropriate technical solutions.

Mapping	Level	Justification
101003/ CS822U.CO1- PSO2	Medium	Involves algorithm design and application of software development practices to implement solutions.
101003/ CS822U.CO1- PSO3	Medium	Applies computer science fundamentals in research or innovative problem solving for societal needs.
101003/ CS822U.CO2- PO1	Medium	Utilizes engineering knowledge to design and develop products or systems.
101003/ CS822U.CO2- PO2	Medium	Requires analysis of user needs and technological feasibility before development.
101003/ CS822U.CO2- PO3	Medium	Focuses on design and development of solutions addressing real-world requirements.
101003/ CS822U.CO2- PO5	Medium	Requires use of modern engineering and IT tools for system or product development.
101003/ CS822U.CO2- PO6	High	Emphasizes sustainability and societal impact during solution development.
101003/ CS822U.CO2- PO7	Medium	Ethical and professional responsibilities are considered when developing socially relevant technologies.
101003/ CS822U.CO2- PO8	Low	Development activities may require collaboration among team members.
101003/ CS822U.CO2- PO9	Low	Involves communicating design ideas and development outcomes.
101003/ CS822U.CO2- PO10	Low	Requires basic planning and coordination during development tasks.
101003/ CS822U.CO2- PO11	Low	Encourages continuous learning of emerging technologies while building solutions.

Mapping	Level	Justification
101003/ CS822U.CO2- PSO1	Medium	Applies computer science fundamentals to design technically feasible solutions.
101003/ CS822U.CO2- PSO2	Medium	Uses programming and software development practices to implement solutions.
101003/ CS822U.CO3- PO8	High	Strongly relates to functioning effectively as a member or leader in multidisciplinary teams.
101003/ CS822U.CO3- PO9	Medium	Requires effective communication among team members during collaboration.
101003/ CS822U.CO3- PO10	Medium	Team leadership involves project coordination and task management.
101003/ CS822U.CO3- PO11	Low	Team interaction supports learning from peers and adapting new approaches.
101003/ CS822U.CO3- PSO3	High	Professional collaboration and teamwork support innovative computer science solutions.
101003/ CS822U.CO4- PO2	Low	Requires analysis and planning before executing project tasks.
101003/ CS822U.CO4- PO3	Medium	Involves designing and implementing project solutions based on planned activities.
101003/ CS822U.CO4- PO5	High	Requires effective use of modern tools and resources during project execution.
101003/ CS822U.CO4- PO6	Medium	Considers societal and environmental impact while implementing solutions.
101003/ CS822U.CO4- PO7	High	Emphasizes adherence to ethical practices and professional responsibilities.
101003/ CS822U.CO4- PO8	Medium	Task execution may involve teamwork and coordination among team members.

Mapping	Level	Justification
101003/ CS822U.CO4- PO9	Medium	Requires communication of progress, outcomes and documentation.
101003/ CS822U.CO4- PO10	High	Focuses on project management including scheduling, resource allocation and task execution.
101003/ CS822U.CO4- PO11	Low	Encourages adaptability and continuous learning during project implementation.
101003/ CS822U.CO4- PSO2	Medium	Uses software development practices to execute and manage project tasks.
101003/ CS822U.CO4- PSO3	Medium	Applies computer science knowledge in practical project development environments.
101003/ CS822U.CO5- PO1	Medium	Uses engineering knowledge to understand technological limitations and opportunities.
101003/ CS822U.CO5- PO2	High	Requires deep problem analysis and identification of research gaps.
101003/ CS822U.CO5- PO3	High	Proposes innovative design solutions to address identified gaps.
101003/ CS822U.CO5- PO4	Low	May involve limited investigation or validation of proposed ideas.
101003/ CS822U.CO5- PO5	Medium	Uses appropriate tools and techniques to analyze and develop innovative solutions.
101003/ CS822U.CO5- PO10	Low	Requires minimal planning while proposing new solution approaches.
101003/ CS822U.CO5- PO11	High	Strongly promotes independent learning and exploration of emerging technologies.
101003/ CS822U.CO5- PSO1	High	Applies computer science fundamentals to identify and solve complex technical problems.

Mapping	**Level**	**Justification**
101003/ CS822U.CO6- PO3	Medium	Requires presentation of design concepts and developed solutions.
101003/ CS822U.CO6- PO6	Medium	Communication may include discussion of societal or environmental implications.
101003/ CS822U.CO6- PO7	Medium	Requires responsible and ethical communication of technical results.
101003/ CS822U.CO6- PO8	Medium	Communication supports collaboration within team environments.
101003/ CS822U.CO6- PO9	High	Strongly relates to effective written and oral communication of technical findings.
101003/ CS822U.CO6- PO10	Low	Requires structured reporting and documentation in project management contexts.
101003/ CS822U.CO6- PO11	Low	Encourages continuous improvement in communication and presentation skills.
101003/ CS822U.CO6- PSO3	High	Communication of innovative computer science solutions and research outcomes.